

Práctica: Filtrado de Imágenes BMP mediante Paleta Cuantizada

Datos Generales

- **Estudiante titular:** Luis Fernando Tlapalcoyoac Olivares

-
- Unidad de Aprendizaje: Multimedia
 - Institución: UPIITA - Instituto Politécnico Nacional (IPN)

1.Introducción

En la presente práctica se llevó a cabo la manipulación a nivel binario de una imagen en formato BMP. El objetivo principal fue alterar la profundidad cromática original para aplicar un filtro basado en una paleta restrictiva de 16 tonalidades de azul. Este procedimiento permitió comprender la estructura interna de los archivos BMP, específicamente el manejo de su cabecera y la codificación de colores en el modelo BGR.

2. Metodología y Desarrollo

Para lograr la transformación de la imagen, se diseñó un algoritmo estructurado en tres fases principales. En primer lugar, se realizó la extracción y preservación de los primeros 54 bytes correspondientes a la cabecera del archivo, garantizando así la integridad estructural de la imagen resultante.

Posteriormente, se construyó una escala de 16 tonalidades azuladas. Esto se logró asignando valores incrementales elevados a la componente azul, valores medios a la verde y valores bajos a la roja, asegurando en todo momento que ningún componente superara el límite máximo de 255.

Finalmente, se ejecutó un recorrido iterativo sobre la matriz de píxeles. Por cada triada de bytes, se determinó el nivel de luminancia promediando sus tres componentes. Dicho promedio se dividió en 16 intervalos para mapear el píxel original a la tonalidad correspondiente dentro de la nueva paleta.

3. Código Fuente

Python

```
# Transformación de imagen a una escala de 16 tonalidades de azul

imagen_entrada = open('./images/volcan.bmp', 'rb')
imagen_salida = open('./images/volcan_azul.bmp', 'wb')

# 1. Extracción y copiado de la cabecera (54 bytes)
cabecera_bmp = imagen_entrada.read(54)
imagen_salida.write(cabecera_bmp)

# 2. Construcción de la paleta de 16 tonalidades de azul (BGR)
escala_azules = []
for nivel in range(16):
    val_r = min(0 + (nivel * 15), 255)
    val_g = min(40 + (nivel * 12), 255)
    val_b = min(80 + (nivel * 11), 255)
    escala_azules.append([int(val_b), int(val_g), int(val_r)])

# 3. Sustitución de píxeles
imagen_entrada.seek(54, 0)
contador_pixeles = 0

while True:
    bytes_pixel = imagen_entrada.read(3)
    if not bytes_pixel:
        break

    b_orig, g_orig, r_orig = bytes_pixel[0], bytes_pixel[1],
    bytes_pixel[2]

    # Determinación de luminancia y mapeo
    nivel_luminancia = (r_orig + g_orig + b_orig) // 3
    idx_color = min(nivel_luminancia // 16, 15)

    pixel_filtrado = escala_azules[idx_color]
    imagen_salida.write(bytes(pixel_filtrado))
```

```
    contador_pixeles += 1

print('Filtrado completado.')
imagen_entrada.close()
imagen_salida.close()
```

4. Resultados Visuales





5. Conclusiones

La práctica demostró exitosamente la viabilidad de manipular directamente los datos binarios de un archivo multimedia. Se observó que la cuantización de colores a un espectro reducido genera una pérdida natural de detalles finos; no obstante, el mapeo basado en la luminancia preservó las formas y volúmenes fundamentales.